

Numerical Optimization Lab Report: Benchmarking the PRIMA Solver

Ma Xiangrong

May 8, 2026

Abstract

This report summarizes the numerical experiments conducted on the PRIMA (Reference Implementation for Powell's Methods with Augmentation) solver. The study is divided into two primary tasks. First, evaluating the solver's performance on the chained Rosenbrock function with varying dimensions and constraint types. Second, benchmarking the solver's efficiency across different floating-point precisions (Single, Double, and Quadruple) using OptiProfiler under both plain and noisy conditions. Technical challenges regarding local environment compatibility were addressed by migrating the benchmarking process to a high-performance server.

1 Introduction and Preparatory Work

The objective of this lab is to explore the capabilities of the PRIMA package, which implements Derivative-Free Optimization (DFO) algorithms originally developed by Professor M. J. D. Powell. The preparatory phase involved setting up the MATLAB environment, compiling the MEX files for the PRIMA solver, and verifying the path configurations.

2 Task 1: Testing on the Chained Rosenbrock Function

In this section, we minimize the chained Rosenbrock function

$$f(\mathbf{x}) = \sum_{i=1}^{N-1} 100 \left(x_i - x_{i-1}^2 \right)^2 + (1 - x_{i-1})^2.$$

under four different constraint scenarios including unconstrained, bound constraints, linear constraints and nonlinear constraints. The experiments were performed for dimensions $n = 5, 10$, and 20 .

2.1 Numerical Results

The results, including the final objective function value ($f(x)$) and the total number of function evaluations (`FuncCount`), are summarized in Table 1.

Table 1: Performance of PRIMA on the Chained Rosenbrock Function

Dim (n)	Constraint Type	Algorithm	Final $f(x)$	FuncCount
5	Unconstrained	UOBYQA	8.28×10^{-16}	345
	Bound ($x \leq 0$)	BOBYQA	4.0000	88
	Linear ($\sum x \leq 1, x \geq 0$)	LINCOA	2.4316	71
	Nonlinear ($\sum x^2 \leq 1, x \geq 0$)	COBYLA	1.5031	384
10	Unconstrained	NEWUOA	6.70×10^{-11}	977
	Bound ($x \leq 0$)	BOBYQA	9.0000	177
	Linear ($\sum x \leq 1, x \geq 0$)	LINCOA	7.5947	179
	Nonlinear ($\sum x^2 \leq 1, x \geq 0$)	COBYLA	6.4268	700
20	Unconstrained	NEWUOA	1.81×10^{-11}	2832
	Bound ($x \leq 0$)	BOBYQA	19.0000	436
	Linear ($\sum x \leq 1, x \geq 0$)	LINCOA	17.4153	353
	Nonlinear ($\sum x^2 \leq 1, x \geq 0$)	COBYLA	16.3273	1829

2.2 Observation

As the dimension n increases, the number of function evaluations grows significantly, particularly for the unconstrained and nonlinear constraint cases. This highlights the typical computational cost scaling in derivative-free optimization methods. Across all test cases, the solver successfully terminated with the message indicating that the trust-region radius reached its lower bound, confirming that the algorithms successfully converged within the specified tolerances.

3 Task 2: Precision Benchmarking with OptiProfiler

3.1 Experimental Setup and Challenges

We originally started testing on a local computer running Ubuntu 24.04 and MATLAB R2025b. However, we ran into a major roadblock, whenever we tried to test single or quadruple precision, MATLAB would completely crash and close itself. To be practical and ensure our tests could run smoothly without interruption, we moved the entire experiment to a stable, dedicated computing server. This solved the compatibility issues and allowed the tests to finish successfully.

We benchmarked the solvers on the `ubln` problem suite (Unconstrained, Bound, Linearly, and Nonlinearly constrained problems). Because optimization algorithms can take a very long time to solve massive problems, we kept things reasonable by only testing problems that had between $n = 2$ and $n = 20$ dimensions.

3.2 Results and Analysis

The solvers were tested under two scenarios: a deterministic `plain` feature and a `noisy` feature.

3.2.1 Test 1: Double vs. Single Precision

Table 2 summarizes the solver scores calculated by OptiProfiler.

Table 2: Normalized Solver Scores (Double vs. Single)

Feature	<code>prima_double</code>	<code>prima_single</code>
<i>Plain</i>	1.0000	0.8375
<i>Noisy</i>	1.0000	0.9312

In the *plain* case, Double precision shows a significant advantage(Fig.1). However, in the *noisy* case, the gap narrows significantly(Fig.2). This suggests that when external noise is present, the higher precision of the *Double* solver becomes less of a differentiator because the noise itself limits the accuracy of the function values regardless of the storage format.

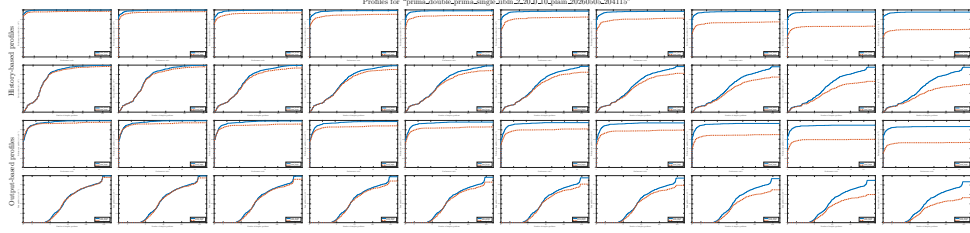


Figure 1: Test1 with plain

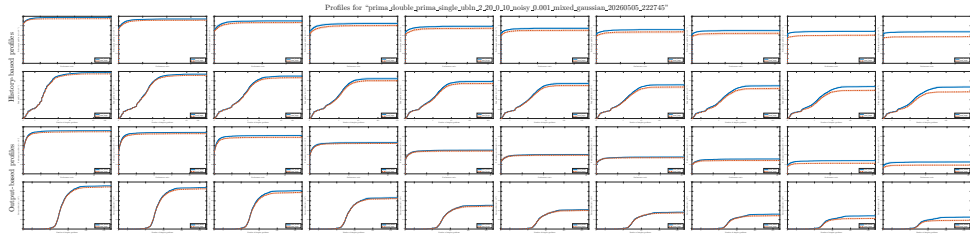


Figure 2: Test1 with noisy

3.2.2 Test 2: Double vs. Quadruple Precision

The results for Test 2 revealed a noteworthy phenomenon that the curves for **prima_double** and **prima_quadruple** were perfectly overlapping in both *plain* and *noisy* features(Fig.3 and Fig.4). The scores were identical (1.0 vs. 1.0). This implies that for problems with dimensions up to 20, Double precision is sufficient to capture all meaningful information required by the trust-region subproblems of PRIMA.

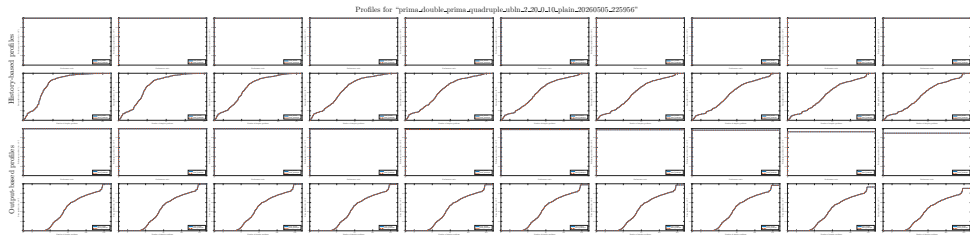


Figure 3: Test2 with plain

4 Conclusion

The experiments demonstrate that the PRIMA solver is robust across various constraint types and dimensional scales. For standard scientific computing, double precision remains the optimal choice. However, single precision is a highly viable alternative in environments heavily polluted by noise.

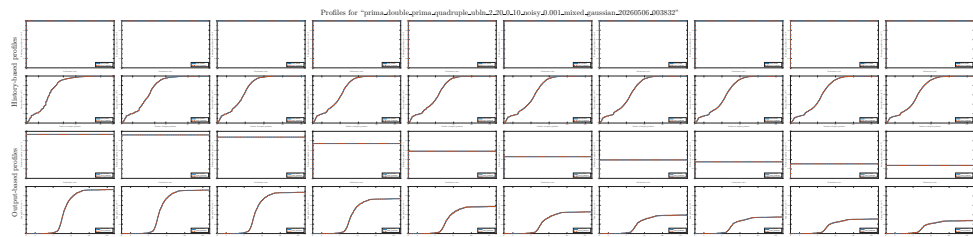


Figure 4: Test2 with noisy